

AstraNotes

Backlog & Sprint Zero Plan

Jonathan Fong · April 9, 2026

Part 1 — User Stories

US-01 As a user, I want to create a note of a specific type so that I can capture information in the right format from the start.

Traces to	<i>FR-1: system shall allow creation of a TextNote, VoiceNote, or SecureNote identified by a system-generated UUID and a user-supplied title — Week 1.2 Initial Requirements</i>
-----------	--

Acceptance Criteria

1	Given the application is running, when I request a new note and supply a type and title, then a note is created with a system-generated UUID and the title I supplied.
2	Given I attempt to create a note with an empty or whitespace-only title, then the operation is rejected, a descriptive failure status is returned, and no note is added to the collection.
3	Given I create a SecureNote, when the note is saved, then no plaintext content is stored in any data member or written to any file.
4	Given NoteFactory is responsible for construction, when a new note type is introduced in a future sprint, then no existing note class — TextNote, VoiceNote, or SecureNote — requires modification.

US-02 As a user, I want to edit the title or body of an existing TextNote so that I can keep information current.

Traces to	<i>FR-2: system shall allow editing of TextNote title or body with automatic last-modified timestamp update — Week 1.2 Initial Requirements</i>
-----------	---

Acceptance Criteria

1	Given an existing TextNote, when I update its title or body, then the edited field is saved, the last-modified timestamp updates to reflect the time of the change, and the created-date field remains unchanged.
2	Given I supply an empty string as a replacement title, then the system rejects the update and returns a failure status without leaving the note in a partially updated state.
3	Given I attempt to edit a note by a UUID that does not exist in the collection, then the system returns a failure status without throwing an exception.

US-03 As a user, I want to delete a note by its UUID so that I can remove notes I no longer need.

Traces to	<i>FR-3: system shall allow deletion by UUID; non-existent UUID returns failure status without exception — Week 1.2 Initial Requirements</i>
-----------	--

Acceptance Criteria	
1	Given a note exists with a known UUID, when I request deletion, then the note is removed from the in-memory collection and is no longer retrievable.
2	Given I supply a UUID that does not exist in the collection, then the operation returns a descriptive failure status without throwing an exception or crashing.
3	Given a deleted note was of type SecureNote, then no plaintext or ciphertext for that note remains accessible in memory after the deletion completes.
4	Given multiple notes exist, when I delete one by UUID, then all other notes remain intact and accessible by their respective UUIDs.

US-04 As a user, I want my notes saved to local storage and restored when I reopen the application so that I do not lose work between sessions.

Traces to	<i>FR-4 (JSON persistence), FR-5 (factory reconstruction on load), NFR-1 (O(1) UUID lookup), SPR-2 (corrupt file quarantine) — Week 1.2 Initial Requirements</i>
Acceptance Criteria	
1	Given I create a note, when the application closes and restarts, then the note appears in the loaded collection with all fields intact: type, UUID, title, body, created-date, and last-modified.
2	Given notes of type TextNote, VoiceNote, and SecureNote exist on disk, when the application loads, then each is reconstructed into its correct concrete type using NoteFactory — no note class requires modification when new types are added.
3	Given a corrupt or unreadable note file exists at startup, when the application loads, then that file is quarantined, all remaining notes load successfully, and the application does not crash or silently discard data.
4	Given the note collection is loaded into memory, when I look up a note by UUID, then the retrieval completes in O(1) average time with no full linear scan of the collection.

US-05 As a user, I want to read the content of a SecureNote only after supplying the correct passphrase so that sensitive information stays private.

Traces to	<i>FR-6 (passphrase-gated access, no plaintext stored), SPR-1 (ciphertext only on disk), SPR-3 (mock-testable modules) — Week 1.2 Initial Requirements</i>
Acceptance Criteria	
1	Given a SecureNote exists, when I supply the correct passphrase, then the plaintext content is decrypted and returned; no plaintext is retained in a persistent data member after the operation ends.
2	Given a SecureNote exists, when I supply an incorrect passphrase, then no content is revealed, a clear failure status is returned, and no partial plaintext is accessible.
3	Given I supply an empty or null passphrase, then the system rejects the access attempt immediately and returns a failure status without attempting decryption.
4	Given a SecureNote is written to disk, then the serialized file contains only ciphertext — no plaintext appears in any file, log, or temporary buffer.
5	Given the EncryptionEngine is defined as an injectable interface, when a unit test for SecureNote is written, then the test can substitute a mock EncryptionEngine without requiring a real crypto library or filesystem access.

US-06 As a user, I want to search my notes by title substring so that I can find a specific note quickly without scrolling through the full list.

Traces to	<i>FR-7: system shall allow search and filter by title substring over the in-memory collection — Week 1.2 Initial Requirements</i>
------------------	--

Acceptance Criteria

1	Given notes exist in the collection, when I search for a substring, then all notes whose titles contain that substring are returned and no others.
2	Given no notes match the search term, then the result is an empty collection with no error, exception, or crash.
3	Given the search is performed on the in-memory collection, then no file I/O is performed during the search operation.
4	Given case-sensitivity is the initial implementation choice, this decision must be recorded as an explicit design decision in planning/ — search behavior must be documented, not silently defaulted.

Part 2 — Prioritized Backlog

Priority	Backlog Item	Story	Depends On	Sprint Target
HIGH	Repository & Folder Structure Create module folders, CMakeLists.txt, README, and planning/ with all four required planning files.	—	—	Sprint 0
HIGH	GTest Scaffold Add googletest to CMake; write one stub test; confirm cmake --build and ctest both exit 0.	—	B-01	Sprint 0
HIGH	NoteManager Interface Draft NoteManager.h with add(), remove(), findByUUID() using unique_ptr; document ownership semantics inline.	US-01	B-01	Sprint 0
HIGH	Note Model Stubs Draft Note.h base class and TextNote.h, VoiceNote.h, SecureNote.h stubs with UUID, title, and metadata fields. No implementations yet.	US-01	B-01	Sprint 0
HIGH	Storage & Encryption Interfaces Draft StorageInterface.h and EncryptionEngine.h as injectable abstractions — no raw dependencies in public API, GTest-mockable.	US-04/05	B-03/04	Sprint 0
HIGH	JSON Library Selection Evaluate and document chosen library with justification: tradeoffs, C++17 compatibility, GTest linkage, license, and supply-chain risk.	—	B-01	Sprint 0
HIGH	Note Creation Implement NoteFactory, UUID generation, note-type assignment, and empty-title rejection to fully satisfy US-01.	US-01	B-03/04/05	Sprint 1

Priority	Backlog Item	Story	Depends On	Sprint Target
HIGH	JSON Persistence & Load Implement FileStorage to serialize and deserialize all note types; reconstruct concrete types via NoteFactory on startup.	US-04	B-07	Sprint 1
HIGH	SecureNote Passphrase Gate Wire EncryptionEngine to SecureNote; enforce encrypted-only persistence with no plaintext stored on disk or in memory.	US-05	B-05/08	Sprint 1
MED	Edit TextNote Implement title and body updates with auto-timestamp, empty-title rejection, and failure status on missing UUID.	US-02	B-07	Sprint 2
MED	Delete Note by UUID Implement deletion with isolation check, SecureNote data cleanup, and failure status on missing UUID.	US-03	B-07	Sprint 2
MED	Corrupt File Quarantine Quarantine unreadable note files at startup; load all remaining notes without crashing or discarding data.	US-04	B-08	Sprint 2
LOW	Title Substring Search Implement in-memory search with empty-result handling, no file I/O, and a documented case-sensitivity decision.	US-06	B-07	Sprint 3

Priority Rationale

- **B-03/04/05** NoteManager.h, Note stubs, and interfaces are Sprint Zero — not Sprint 1 —
 - because every Sprint 1 feature task depends on them. If NoteManager.h is missing at Sprint 1 start, there is no stable interface to implement against. If EncryptionEngine.h is undefined when FileStorage is written, the persistence layer must be partially rewritten to satisfy SPR-1. Front-loading interfaces eliminates both risks before implementation begins.
- **B-05** EncryptionEngine.h is Sprint Zero despite being a SecureNote concern —
 - because the no-plaintext-on-disk constraint (SPR-1) affects how FileStorage serializes SecureNote. FileStorage cannot be written correctly without knowing the encryption boundary. An undefined interface at Sprint 1 start means persistence code will assume or invent behavior — requiring rework when the real interface is defined.
- **B-07/10/11** Create-note and persistence are HIGH; edit and delete are MEDIUM —
 - because you cannot edit or delete a note that cannot be created or persisted. Create and persist are foundational — removing either makes all remaining stories untestable. Edit and delete have no blocking downstream dependencies of their own and follow cleanly in Sprint 2 without rework.
- **B-13** Search (US-06) is LOW —
 - because it requires no new architecture. It operates on the in-memory collection established by Sprint 1, introduces no security concern, and can be added in Sprint 3 without touching any earlier module.
- **B-06** JSON library selection is HIGH in Sprint Zero —

- because NFR-4 requires a written justification for any external library before adoption. Skipping the compatibility check risks a build or linkage failure that stalls Sprint 1. Resolving selection in Sprint Zero means Sprint 1 can use the library from day one with confidence.

Part 3 — Sprint Zero Plan

Goal: Produce a stable build environment, reviewed interface drafts, and complete planning artifacts so that Sprint 1 can begin feature work without unresolved technical or planning debt.

Sprint Zero Tasks

#	Task	Category	DoD Exit Gate
S0-1	Create repo. Folder structure: model/, controller/, storage/, encryption/, test/. Add CMakeLists.txt, .gitignore, README with setup instructions. Create planning/ with requirements.md, user-stories.md, backlog.md, sprint-zero-plan.md.	Setup	Repo builds; README present; planning/ folder populated with four files
S0-2	Configure GTest. Add googletest as a CMake dependency. Write one stub test that compiles and passes. Run cmake --build and ctest to confirm the full build-test loop works before any implementation begins.	Setup	cmake --build && ctest both exit 0; one passing test confirmed
S0-3	Draft NoteManager.h. RAII-based interface: add(), remove(), findByUUID(). Use unique_ptr<Note> throughout. Document ownership choice and any shared_ptr exceptions inline in the header.	Technical	All public methods documented; no raw pointer ownership in public API; DoD header-file criteria met
S0-4	Draft Note.h base class plus TextNote.h, VoiceNote.h, SecureNote.h stubs. Include: UUID field, title, created_at, last_modified_at, type indicator. No implementations yet — stubs only.	Technical	All four headers compile; all Week 1.2 metadata fields present
S0-5	Draft StorageInterface.h and EncryptionEngine.h as injectable abstract interfaces. Goal: unit tests must be able to substitute a mock for either without a real filesystem or real crypto library.	Technical	No external library dependencies in public API; both interfaces mockable under GTest
S0-6	Evaluate and select a JSON serialization library (candidate: nlohmann/json). Write justification in planning/library-decision.md: capability gap vs. hand-rolled, C++17 compatibility, GTest linkage test, license, supply-chain risk.	Technical	planning/library-decision.md committed; NFR-4 obligation satisfied
S0-7	Confirm requirement traceability. Verify all 6 user stories map to at least one FR/NFR/SPR from the Week 1.2 document. Record any gap, ambiguity, or open question in planning/traceability.md.	Planning	Trace table complete; no orphan stories; open questions documented
S0-8	Initialize prompt log per Working Agreement format: date, task, prompt summary, output summary, decision, rationale. Log every AI interaction from Sprint Zero before the S0-10 review.	Workflow	At least one complete log entry per AI session; format matches Working Agreement spec

#	Task	Category	DoD Exit Gate
S0-9	Complete risk log with at least four entries. Each entry must include: risk description, likelihood (H/M/L), impact (H/M/L), and mitigation. Commit to planning/risk-log.md.	Risk	<i>Four entries in planning/risk-log.md; all fields populated; no entry is vague</i>
S0-10	Sprint Zero review. Walk all nine prior tasks against the Definition of Done checklist. Confirm no task is vague, incomplete, or unexplainable. Record any deferred item with an explicit rationale.	Workflow	<i>DoD checklist signed off; any deferred item has a written rationale; Sprint 1 can start</i>

Sprint Zero Exit Criteria

Sprint Zero is complete when all of the following are true. If any criterion cannot be met, the blocker is documented immediately — Sprint Zero does not slip silently into Sprint 1.

- ✓ Repo is live. Folder structure matches AstraNotes module layout. cmake --build exits 0.
- ✓ GTest scaffold in place. ctest exits 0 with at least one passing test.
- ✓ NoteManager.h, Note.h, TextNote.h, VoiceNote.h, SecureNote.h drafted and compiling. All public methods documented.
- ✓ StorageInterface.h and EncryptionEngine.h drafted as injectable interfaces. No external dependencies in public API.
- ✓ JSON library selected and justified in planning/library-decision.md. C++17 compatibility and GTest linkage confirmed.
- ✓ Requirement traceability table complete. All 6 user stories mapped. No orphan stories.
- ✓ Prompt log initialized with at least one entry per AI session in Sprint Zero.
- ✓ Risk log contains at least four entries with likelihood, impact, and mitigation.
- ✓ Sprint Zero DoD review completed. All items cleared or any blocker documented with rationale.

Risk Log

Risk	Like.	Impact	Mitigation
EncryptionEngine boundary undefined before persistence work	H	H	S0-5 drafts EncryptionEngine.h this sprint. No FileStorage code touching SecureNote may be written until this interface is reviewed. If the gate is missed, SPR-1 compliance requires rework in Sprint 1.
JSON library C++17 / GTest linkage conflict	M	H	S0-6 requires a build-confirmed linkage check before adopting any library. If the candidate fails, the fallback is a minimal hand-rolled JSON serializer scoped to the AstraNotes note schema, documented in planning/library-decision.md as a contingency.
NoteManager ownership semantics chosen incorrectly	L	H	unique_ptr is the initial choice for single ownership. If shared ownership is needed later, the interface change is an explicit design decision, not a silent leak. Document the unique_ptr rationale inline in NoteManager.h so it is traceable and revisable.
Scope creep from extensibility points	M	M	AstraNotes has natural extension points (new note types, plugin interfaces). None enter Sprint Zero. Working Agreement scope discipline: no feature enters the backlog without a charter requirement mapping or professor-directed change. S0-10 reviews the backlog for drift.

Part 4 — AI Use Reflection

AI helped structure this planning document by translating the Week 1.2 requirements into a coherent set of user stories, acceptance criteria, a prioritized backlog, and a Sprint Zero plan. Rather than starting from a blank page, the process began with the requirements already defined and used AI to organize them into a practical planning layer that the course asked for.

The user stories capture the most important early functionality — note creation, editing, deletion, persistence, secure access, and search — in a format that is clear and traceable back to the original requirements. The acceptance criteria are written in Given/When/Then format so each one maps directly to something testable. The backlog is sequenced around what must exist before other work can begin, with Sprint Zero focused entirely on foundations: project setup, core interfaces, and key technical decisions rather than features. The result is a plan that is realistic for a single-developer quarter project and grounded in the actual AstraNotes charter rather than generic software planning patterns.